

Frequency Analysis with Index of Coincidence

CS 5158 6058 Project 1

Student	BENJAMIN ISHIMWE
UCID	M17996246
Course	[CS 5158 or CS 6058]
Operating system	macOS [version from About This Mac]
Programming language	Python [version from python3 --version]

This report explains my implementation of English letter-frequency analysis, the Vigenere cipher, an Index of Coincidence attack, and the batch attack on ciphertexts from the class shared folder. The screenshots below must be replaced with the outputs I capture while running the included program on my laptop.

Environment Build and Program Use

I developed and ran the project on macOS [insert version]. I used Python [insert the exact output of python3 --version]. Python does not compile into C/C++ object files. To follow the required directory structure, my build script packages all source code from src into the runnable executable archive build/ic_project.pyz.

I installed the dependency and built the program from the project root with:

```
python3 -m pip install -r requirements.txt
python3 build_project.py
```

The build command places the executable only in the build folder. I then use the same command pattern for every function: python3 build/ic_project.pyz followed by the function name and its arguments. The data folder contains all inputs and all results generated by the program.

Task 1 English Frequency and Index of Coincidence

I ran the frequency analysis from the project root with this command:

```
python3 build/ic_project.pyz frequency --input data/english_source.txt
```

My text source was Pride and Prejudice by Jane Austen, Project Gutenberg eBook 1342. The original source is <https://www.gutenberg.org/ebooks/1342>. After removing digits, spaces, punctuation, and other nonalphabetic symbols, the program analyzed 563,984 lowercase English letters. The frequency function counted every letter from a through z.

I calculated IC as the sum of $f_i(f_i - 1)$ for all 26 letters divided by $N(N - 1)$, where f_i is a letter count and N is the number of retained letters. My result was 0.065385. This is close to

the expected English value of about 0.066, so the result is consistent with the value discussed in class. A small difference is expected because letter frequencies depend on the selected text.

[Screenshot 1 frequency table in the VS Code terminal - insert the screenshot captured on your laptop]

[Screenshot 2 frequency plot open in VS Code - insert the screenshot captured on your laptop]

[Screenshot 3 IC result in the VS Code terminal - insert the screenshot captured on your laptop]

Task 2 Vigenere Encryption and Decryption

I used the key bearcats, which has eight letters. The program retained exactly the first 100 lowercase letters of my input:

```
cryptographyprotectsmessagesbytransformingreadableinformationintocodedtextthatunauthorizedpeoplecann
```

I encrypted the text with:

```
python3 build/ic_project.pyz encrypt --input data/task2_plaintext.txt --key bearcats --limit 100 --output data/task2_ciphertext.txt
```

For each plaintext letter and repeating key letter, encryption adds their values modulo 26. The program produced the following 100 uppercase letters:

```
DVYGV0ZJBTHPRRHLFGTJOELKBKEJDYMBRSWQRF AOKRVCDDTTMII EHOKEBXIFPIGLPGOUGDMWXYCTNFBYTYQ  
RBRFHPVQPEWDENE
```

[Screenshot 4 encryption command and output - insert the screenshot captured on your laptop]

I decrypted the ciphertext with:

```
python3 build/ic_project.pyz decrypt --input data/task2_ciphertext.txt --key bearcats --limit 100 --output data/task2_decrypted.txt
```

Decryption subtracts each repeating key value modulo 26. The recovered lowercase text matched the original 100 letters exactly.

[Screenshot 5 decryption command and output - insert the screenshot captured on your laptop]

Task 3 Attack with Index of Coincidence

I generated the long uppercase ciphertext with:

```
python3 build/ic_project.pyz encrypt --input data/english_source.txt --key bearcats --output data/m17996246_ishimwe.txt
```

The resulting ciphertext contains 563,984 letters, which is greater than the required minimum of 10,000. I ran the attack with:

```
python3 build/ic_project.pyz attack --input data/m17996246_ishimwe.txt --english-ic data/english_ic.txt
```

For every possible key length from 3 through 19, I separated the ciphertext into columns and calculated the average column IC. The candidate length 8 had average IC 0.065385, which matched the English IC. Length 16 also had the same average IC because it is a multiple of 8, so the program applies a small length penalty and selects the shorter fundamental period.

IC identifies the likely key length but cannot identify a Caesar shift because shifting letters does not change IC. I therefore tested all 26 shifts in each column with chi-square comparison against standard English frequencies. The smallest score in each position produced b, e, a, r, c, a, t, and s. The recovered key length was 8 and the recovered key was bearcats, which exactly matched my key.

[Screenshot 6 IC evidence for candidate key lengths - insert the screenshot captured on your laptop]

[Screenshot 7 key-letter scores and recovered key - insert the screenshot captured on your laptop]

Task 4 Shared Ciphertext Attack

I uploaded my uppercase ciphertext file, named m17996246_ishimwe.txt, to the class shared folder. It contains 563,984 uppercase letters and uses the key bearcats. I then downloaded every class .txt file into data/shared_ciphertexts and ran:

```
python3 build/ic_project.pyz attack-folder --folder data/shared_ciphertexts --english-ic data/english_ic.txt
```

The program generated data/task4_results.txt in the required format:

[Paste every line from data/task4_results.txt after downloading all class files]

[Screenshot 8 Task 4 batch summary in the VS Code terminal - insert the screenshot captured on your laptop]

Conclusion

The experiment showed that English has a nonuniform distribution and an IC above random text. A Vigenere cipher hides the frequencies in the complete ciphertext, but splitting it at the correct period creates Caesar-shifted English columns. Average IC reveals the period, and English-frequency scoring reveals the shifts and the complete key.